

Indice

Esame ML — I 4 errori del pretest (parte individuale)	1
1. Backpropagation — derivazione per l'hidden layer	1
Setup (da fissare a voce prima di derivare)	1
Struttura generale (vale per ogni peso)	2
Caso output ($t = k$) — il “facile”, da citare come confronto	2
Caso hidden ($t = j$) — quello che ti è stato chiesto	2
Lettura a voce (cosa dire all'esame)	3
Possibili domande di rimbalzo	3
2. SVM hard margin — formulazione primale	3
Setup	3
Da dove esce il “1”: iperpiano canonico	4
Formulazione primale (hard margin)	4
Lettura dei pezzi (cosa dire)	4
Possibili domande di rimbalzo	4
3. RNN — caratteristiche principali (elenco)	4
4. Rappresentabilità: una shallow network può eguagliare una DNN?	5
Caso A — shallow = un solo hidden layer (non lineare), con abbastanza unità → Sì, stessa rappresentabilità della DNN	6
Il “MA” che fa il voto — rappresentabilità $\neq$ efficienza	6
Caso B — “banale” <i>davvero</i> = nessun hidden layer / attivazioni lineari → NO	6
Sintesi da dire a voce	6
Condizioni per l'universalità (riassunto)	7
Possibili domande di rimbalzo	7

Esame ML — I 4 errori del pretest (parte individuale)

Notazione coerente col corso (Micheli / Notation summary): $E_p = \frac{1}{2} \sum_{k=1}^K (d_k - o_k)^2$, $net_t = \sum_s w_{ts} o_s$, $o_t = f_t(net_t)$, $\delta_t = -\partial E_p / \partial net_t$. In SVM: target $d \in \{-1, +1\}$, bias b , N dati, m feature.

Indice: 1. Backpropagation — derivazione hidden layer ($\partial E_p / \partial o_j$, caso $t = j$) 2. SVM hard margin — primale: formulazione e vincoli 3. RNN — caratteristiche principali 4. Rappresentabilità: una rete “banale” può eguagliare una DNN?

1. Backpropagation — derivazione per l'hidden layer

Setup (da fissare a voce prima di derivare)

- Rete feed-forward fully connected (MLP). Indici per i layer: i (input), j (hidden), k (output).
- Errore su un singolo pattern p (la p si omette per alleggerire):

$$E_p = \frac{1}{2} \sum_{k=1}^K (d_k - o_k)^2$$

- Per ogni unità t : $net_t = \sum_s w_{ts} o_s$ (somma pesata degli input), $o_t = f_t(net_t)$ (attivazione).
- Definizione di **delta** (segnale d'errore dell'unità t): $\delta_t = -\frac{\partial E_p}{\partial net_t}$.

Struttura generale (vale per ogni peso)

$$\Delta_p w_{tu} = -\frac{\partial E_p}{\partial w_{tu}} = -\frac{\partial E_p}{\partial net_t} \cdot \frac{\partial net_t}{\partial w_{tu}} = \delta_t \cdot o_u$$

perché $\frac{\partial net_t}{\partial w_{tu}} = \frac{\partial \sum_s w_{ts} o_s}{\partial w_{tu}} = o_u$ (tutti i termini sono 0 tranne $s = u$).

Espandendo il delta con la chain rule (passando per o_t):

$$\delta_t = -\frac{\partial E_p}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot \frac{\partial o_t}{\partial net_t} = -\frac{\partial E_p}{\partial o_t} \cdot f'_t(net_t)$$

Tutto si riduce a calcolare $-\frac{\partial E_p}{\partial o_t}$. Qui si separano i due casi.

Caso output ($t = k$) — il “facile”, da citare come confronto

o_k compare direttamente in E_p :

$$-\frac{\partial E_p}{\partial o_k} = -\frac{\frac{\partial}{\partial} \sum_{r=1}^K (d_r - o_r)^2}{\partial o_k} = (d_k - o_k) \Rightarrow \boxed{\delta_k = (d_k - o_k) f'_k(net_k)}$$

Caso hidden ($t = j$) — quello che ti è stato chiesto

Il punto chiave concettuale: o_j **non compare direttamente** in E_p . E_p dipende solo dalle uscite o_k . L'unità hidden j influenza l'errore **solo indirettamente**, attraverso *tutte* le unità di output k a cui è connessa. Quindi serve la chain rule **multivariata**: si somma su tutti i k .

$$-\frac{\partial E_p}{\partial o_j} = \sum_{k=1}^K \left(-\frac{\partial E_p}{\partial net_k} \right) \frac{\partial net_k}{\partial o_j}$$

Due osservazioni che chiudono la derivazione:

1. $\frac{\partial net_k}{\partial o_j} = \frac{\partial \sum_s w_{ks} o_s}{\partial o_j} = w_{kj}$ (sopravvive solo $s = j$).
2. $-\frac{\partial E_p}{\partial net_k} = \delta_k$ — **lo abbiamo già calcolato** nel caso output! È il cuore del “back-propagation”: riusciamo i δ_k .

Quindi:

$$-\frac{\partial E_p}{\partial o_j} = \sum_{k=1}^K \delta_k w_{kj}$$

e rimettendo la derivata dell'attivazione:

$$\delta_j = \left(\sum_{k=1}^K \delta_k w_{kj} \right) f'_j(\text{net}_j) \quad \Delta_p w_{ji} = \delta_j \cdot o_i$$

Lettura a voce (cosa dire all'esame)

- “ o_j entra nell'errore solo tramite le unità a cui manda segnale, perciò la derivata totale è una **somma sui k** (chain rule multivariata).”
- “ $\partial \text{net}_k / \partial o_j = w_{kj}$: il peso che collega $j \rightarrow k$.”
- “ $-\partial E_p / \partial \text{net}_k = \delta_k$ è già noto: i delta dell'output **si propagano all'indietro** pesati da w_{kj} . Questo è il senso di *backpropagation*.”
- Generalizzazione: con più hidden layer, la somma è su **tutte le unità del layer immediatamente successivo** a cui j è connessa (non per forza output). I delta arrivano dal layer sopra.

Possibili domande di rimbalzo

- **Perché la somma su k e non un singolo termine?** Perché o_j ha effetto su E_p lungo molti percorsi (uno per ogni output connesso): la derivata totale somma i contributi.
- **Quanto vale f'_j per la sigmoide?** $f(\text{net}) = \frac{1}{1+e^{-\text{net}}} \Rightarrow f' = f(1-f) = o_j(1-o_j)$. (Per tanh: $1-o_j^2$.)
- **Bias?** Trattato come peso w_{t0} con input fisso $o_0 = 1$; stessa formula.
- **Costo / “magic factorization”:** δ_t si calcola una sola volta per unità e si riusa per tutti i suoi pesi $\rightarrow$ costo lineare nel numero di pesi, non quadratico.
- **Online vs batch:** $w_{tu}^{\text{new}} = w_{tu} + \eta \delta_t o_u$ per pattern (online); in batch si somma $\Delta_p w$ sui pattern.
- **Trabocchetto di lessico (Micheli ci tiene):** non si dice “backprop NN”. La NN è un *modello*; la backpropagation è l'*algoritmo di training* (calcolo del gradiente propagando gli errori).

2. SVM hard margin — formulazione primale

Setup

Dati linearmente separabili $\{(\mathbf{x}_i, d_i)\}_{i=1}^N$, con $d_i \in \{-1, +1\}$. Iperpiano separatore $\mathbf{w}^\top \mathbf{x} + b = 0$. Funzione di decisione $h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$.

Da dove esce il “1”: iperpiano canonico

La distanza (margine geometrico) di un punto dall'iperpiano è $\frac{|\mathbf{w}^\top \mathbf{x} + b|}{\|\mathbf{w}\|}$. C'è una libertà di scala ($\mathbf{w}, b$ e $\alpha\mathbf{w}, \alpha b$ danno lo stesso iperpiano). La si fissa imponendo la **forma canonica**: per i punti più vicini $|\mathbf{w}^\top \mathbf{x} + b| = 1$. Con questa scelta il **margine** vale $\frac{2}{\|\mathbf{w}\|}$.

Formulazione primale (hard margin)

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{soggetto a} \quad d_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N$$

Lettura dei pezzi (cosa dire)

- **Obiettivo** $\frac{1}{2} \|\mathbf{w}\|^2$: massimizzare il margine $\frac{2}{\|\mathbf{w}\|}$ equivale a minimizzare $\|\mathbf{w}\|$, equivale a minimizzare $\frac{1}{2} \|\mathbf{w}\|^2$ (il $\frac{1}{2}$ e il quadrato sono per comodità: funzione convessa, differenziabile, problema di **programmazione quadratica**). **Nota: b non compare nell'obiettivo.**
- **Vincoli**: uno **per ogni pattern**. $d_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ codifica due cose insieme: (a) classificazione corretta (il segno di $\mathbf{w}^\top \mathbf{x}_i + b$ deve concordare con d_i , quindi il prodotto ≥ 0) e (b) margine ≥ 1 nella scala canonica (quindi ≥ 1 , non solo ≥ 0).
- **Support vector**: i punti che soddisfano il vincolo **con uguaglianza** ($d_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$), cioè quelli sul margine. Sono gli unici che determinano la soluzione.
- **Natura del problema**: ottimizzazione **convessa** vincolata (obiettivo quadratico convesso, vincoli lineari) $\rightarrow$ minimo **globale unico**.
- **“Hard” margin**: assume separabilità lineare perfetta, **nessuna variabile di slack**. Se i dati non sono separabili il problema è infeasible $\rightarrow$ si passa al soft margin (slack ξ_i , parametro C).

Possibili domande di rimbalzo

- **Differenza hard/soft**: soft aggiunge $\xi_i \geq 0$: vincoli $d_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$, obiettivo $\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i$. C regola il trade-off margine/violazioni (bias-variance).
- **Perché il duale?** Il duale (Lagrangiana, moltiplicatori α_i) dipende solo dai prodotti scalari $\mathbf{x}_i^\top \mathbf{x}_j \rightarrow$ **kernel trick**; e i $\alpha_i > 0$ identificano i support vector.
- **Margine in funzione di $\mathbf{w}$** : $2/\|\mathbf{w}\|$. Da derivare se chiesto: i due iperpiani $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ distano $2/\|\mathbf{w}\|$.
- **Perché massimizzare il margine?** Migliore generalizzazione (controllo della capacità / VC dimension, SLT): tra infiniti separatori si sceglie il più robusto.

3. RNN — caratteristiche principali (elenco)

Le **Recurrent Neural Networks** trattano domini diversi dai vettori a dimensione fissa. Caratteristiche principali da elencare:

1. **Dominio sequenziale / temporale.** Input, output e target sono *sequenze* / serie temporali a lunghezza variabile (notazione corso: input $l(t)$, output $o(t)$ o $y(t)$, target $d(t)$, stato $x(t)$). Non più vettori statici di dimensione fissa.
2. **Connessioni ricorrenti (feedback / cicli).** A differenza delle feed-forward, l'architettura ha cicli: lo stato/uscita al tempo t rientra come input al tempo $t + 1$. Non è un DAG aciclico.
3. **Stato interno = memoria.** Mantengono uno stato $x(t)$ che riassume il passato: $x(t) = f(x(t-1), \text{input}(t))$. Questo dà **context-sensitivity**: l'output dipende non solo dall'input corrente ma dalla storia.
4. **Condivisione dei pesi nel tempo (parameter sharing).** Gli **stessi** pesi sono applicati ad ogni passo temporale. Conseguenze: si gestiscono sequenze di lunghezza arbitraria e si generalizza tra posizioni diverse; meno parametri.
5. **Stazionarietà / invarianza temporale.** La funzione di transizione è la stessa ad ogni t (assunzione di sistema tempo-invariante).
6. **Training con Backpropagation Through Time (BPTT).** Si "srotola" (unfold) la rete nel tempo ottenendo una rete feed-forward profonda con pesi condivisi, poi si applica la backpropagation; i gradienti dei pesi condivisi si sommano sui passi temporali.
7. **Problema del gradiente che svanisce/esplode.** Su sequenze lunghe i prodotti di molti termini fanno svanire o esplodere il gradiente → difficoltà ad apprendere **dipendenze a lungo termine**. Motiva le architetture **gated**: LSTM, GRU (gate che regolano cosa ricordare/dimenticare).
8. **Sistema dinamico / espressività.** Una RNN è un sistema dinamico; in teoria molto espressiva (può approssimare sistemi dinamici, Turing-completa in condizioni ideali).
9. **Causalità e modalità I/O.** L'output a t dipende dagli input fino a t (RNN standard, causale). Modalità: sequence-to-sequence, sequence-to-one (es. classificazione di una sequenza), one-to-sequence. (Le bidirezionali rilassano la causalità usando anche il futuro.)

Sintesi in una frase: *RNN = rete con feedback e stato interno, pesi condivisi nel tempo, addestrata con BPTT, pensata per sequenze, con il limite del vanishing gradient risolto in pratica da LSTM/GRU.*

4. Rappresentabilità: una shallow network può eguagliare una DNN?

Inquadramento (dal pretest): la domanda confronta una **shallow network** (un solo hidden layer) con una **DNN** (deep, molti hidden layer). La risposta centrale è il **Caso A**. Il Caso B serve solo se l'esaminatore spinge verso "e se fosse ancora più banale?".

Risposta breve: **SÌ**, sul piano della rappresentabilità sono equivalenti (approssimazione universale) — ma vanno distinte *rappresentabilità, efficienza e apprendibilità*.

Caso A — shallow = un solo hidden layer (non lineare), con abbastanza unità → SÌ, stessa rappresentabilità della DNN

Per il **Teorema di Approssimazione Universale** (Cybenko 1989, Hornik 1991), una rete feed-forward con **un singolo hidden layer**, attivazioni **non lineari** (es. sigmoidi / non polinomiali) e un numero **sufficientemente grande** di unità nascoste può approssimare *qualsiasi* funzione continua su un compatto con precisione arbitraria. Quindi, in termini di *classi di funzioni rappresentabili*, una rete shallow ha lo **stesso potere espressivo** di una DNN: entrambe sono approssimatori universali.

Il “MA” che fa il voto — rappresentabilità $\neq$ efficienza

1. **Costo in larghezza — i “no-flattening theorems” (terminologia del corso, slide DNN 38-40).** “Stessa rappresentabilità” non significa “stesso costo”. Esistono funzioni **composizionali** che una *deep* NN implementa bene e che **non** possono essere implementate mantenendo la stessa efficienza **appiattendo** la rete (*flattening* = ridurre la profondità): es. si passa da un numero **lineare** a un numero **esponenziale** di unità. (In letteratura: risultati di *depth separation*, es. Telgarsky 2016, Eldan-Shamir 2016.) La **profondità** dà quindi un guadagno esponenziale di *efficienza* (a parità di funzione, meno unità): *stessa cosa* è rappresentabile, ma non lo stesso *quanto costa*.
 - **Caveat decisivo (slide 40):** *no guarantee that your task shares such property.* I no-flattening theorems garantiscono il vantaggio della profondità **solo** per funzioni composizionali; non c'è garanzia che il problema specifico abbia quella struttura. È tuttora area di ricerca teorica aperta.
 - **Attenzione al termine:** *flattening* (appiattare la rete, ridurre la profondità), **non flattening**.
2. **Apprendibilità $\neq$ rappresentabilità.** Anche se la funzione è rappresentabile, non è detto che la **discesa del gradiente** trovi quei pesi, né che la rete **generalizzi**. Esistenza dei pesi $\neq$ raggiungibilità in training.

Caso B — “banale” davvero = nessun hidden layer / attivazioni lineari → NO

Se per “la più banale possibile” intendi un **percettrone / LTU senza hidden layer**, oppure una rete con **sole attivazioni lineari**, allora **NO**: - Una rete con sole unità lineari, **per quanto profonda, collassa in una singola trasformazione lineare** ($W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$). Può rappresentare **solo funzioni lineari** → non risolve nemmeno lo **XOR**, strettamente meno di una DNN. - La **non linearità** dell'attivazione è la condizione necessaria: senza, la profondità non aggiunge nulla.

Sintesi da dire a voce

“Sì. Per il **teorema di approssimazione universale**, una shallow con un hidden layer, attivazione non lineare e abbastanza unità approssima qualsiasi funzione continua su un compatto a precisione arbitraria — stessa classe di una DNN. La differenza è in **efficienza**, non in rappresentabilità: per i **no-flattening theorems** del corso, *appiattare* la rete su funzioni composizionali può richiedere un numero **esponenziale** di unità. Con il caveat che non c'è garanzia che un task generico sia composizionale. E rappresentabilità non implica apprendibilità: il teorema garantisce che i pesi esistono, non che il gradiente li trovi.”

Condizioni per l'universalità (riassunto)

Almeno un hidden layer · attivazione non lineare (non polinomiale) · larghezza non limitata.
Mancando la non linearità o l'hidden layer, l'universalità cade.

Possibili domande di rimbalzo

- **Perché lineare+lineare = lineare?** Mostra $W_2 W_1 = W$.
- **Esempio di funzione che richiede non linearità:** XOR.
- **Cosa dà la profondità se l'espressività al limite è uguale?** Efficienza (meno parametri), miglior bias induttivo per funzioni composizionali, spesso migliore generalizzazione.
- **Cosa NON dice il teorema di approssimazione universale (i tre "non"):** è un teorema di *esistenza non costruttivo*. (1) **Non costruttivo:** garantisce che esista una rete/pesi che approssima, ma non fornisce algoritmo o costruzione per trovarli. (2) **Non quantifica la larghezza:** non dice quante unità servano (potrebbero essere un numero impraticabile). (3) **Non garantisce apprendibilità né generalizzazione:** la discesa del gradiente può non raggiungere quei pesi. → *rappresentabilità* $\neq$ *apprendibilità*.